Lecture-4

Node.js Basics

Architecture, modules, the HTTP server, and the File System module



Architecture



HTTP Server



Modules



File System

The traditional server model

One thread handles one request at a time — e.g. classic Java/Tomcat servers



Thread-per-request

Each incoming client gets its own dedicated thread. That thread blocks — waiting on the database, disk, or network — until the work finishes, then sends the response.

More concurrent clients → more threads → more memory & context-switching overhead.



Node.js: single-threaded, non-blocking, asynchronous

One event-loop thread accepts everyone; heavy I/O is handed off to a worker pool



Non-blocking I/O

All clients are accepted by the same single main thread (the event loop). It never blocks — it delegates slow work (file/DB/network) to a small worker pool and moves on to the next client immediately.

When the worker finishes, a callback fires and the response goes out — this is what "asynchronous" means here.



Your first Node.js HTTP server

The built-in http module — no frameworks needed

```
const http = require('http');

const server = http.createServer((req, res) => {
    res.write("Welcome to nodeJs !");
    res.end();
}).listen(9000);

console.log("Hello World");
```

`require('http')` → loads Node's built-in HTTP module.
`createServer((req, res) => {...})` → runs once per incoming request.
`.listen(9000)` → starts listening for connections on port 9000.

Modules: require() and module.exports

Every .js file in Node is its own module, private by default



calc.js

```
function add(a, b) {
  return a + b;
}
module.exports.add = add;
```



app.js

```
var calc = require('./calc.js');
var result = calc.add(67, 76);
console.log("output", result);
// output 143
```

module.exports decides what leaves the file; require(path) pulls that exact object into the requiring file. Everything else inside calc.js stays private.

File System module — Reading a file

fs.readFile() reads a file's contents asynchronously



```
const fs = require('fs');

fs.readFile('calc.js', 'utf8', function (err, data) {
    console.log(data);
});
```

`'calc.js'` → path of the file to read.
`'utf8'` → encoding, so data comes back as a readable string (not raw bytes).
`callback(err, data)` → runs later, once the disk read completes — this call does NOT block the rest of the script.

File System module — Writing a file

fs.writeFile() creates the file if missing, or overwrites it completely if it exists



```
const fs = require('fs');

fs.writeFile('calc.js', 'console.log("hello")', function (err) {
    console.log("data saved");
});
```

Caution: writeFile() replaces the ENTIRE existing content of calc.js with the new string — the old add() function would be gone.

Use this when you want to create a fresh file or fully overwrite one — not for adding to existing content.

File System module — Appending to a file

fs.appendFile() adds to the end of a file without touching what's already there



```
const fs = require('fs');

fs.appendFile('calc.js', 'console.log("hello")', function (err)
{
    console.log("data saved");
});
```

`writeFile` overwrites everything. `appendFile` keeps the original content and adds the new text after it — safer for logs or incremental writes.

If the file doesn't exist yet, appendFile() creates it — same as writeFile in that case.

Recap

What we covered today



Thread-per-request

Traditional servers block one thread per client



Non-blocking event loop

Node handles everyone on one thread + worker pool



HTTP server

http.createServer + .listen(port)



Modules

require() + module.exports connect files



fs.readFile / writeFile

Async read, and full overwrite on write



fs.appendFile

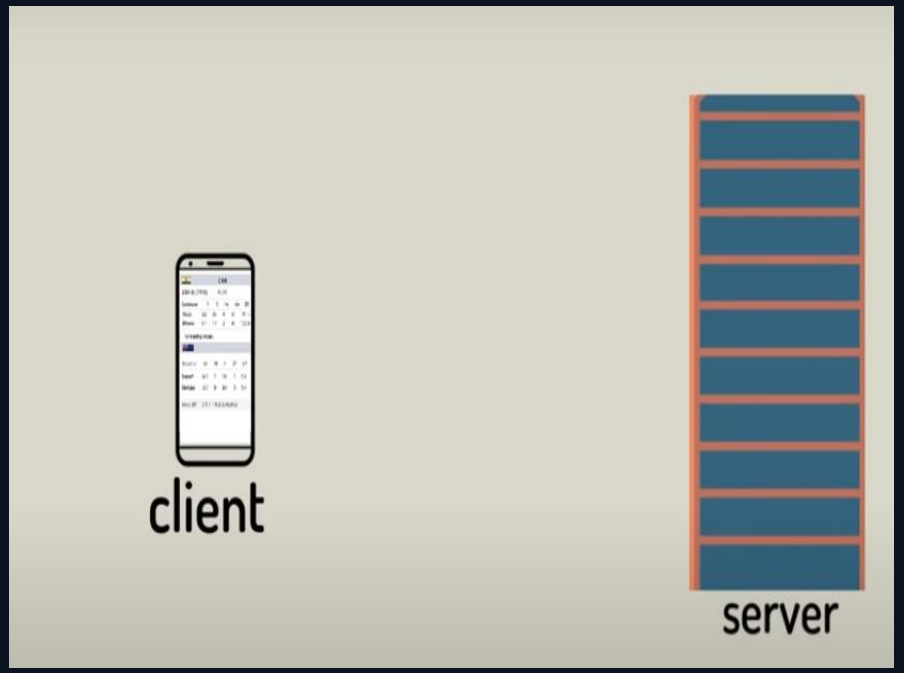
Adds to a file without erasing existing content

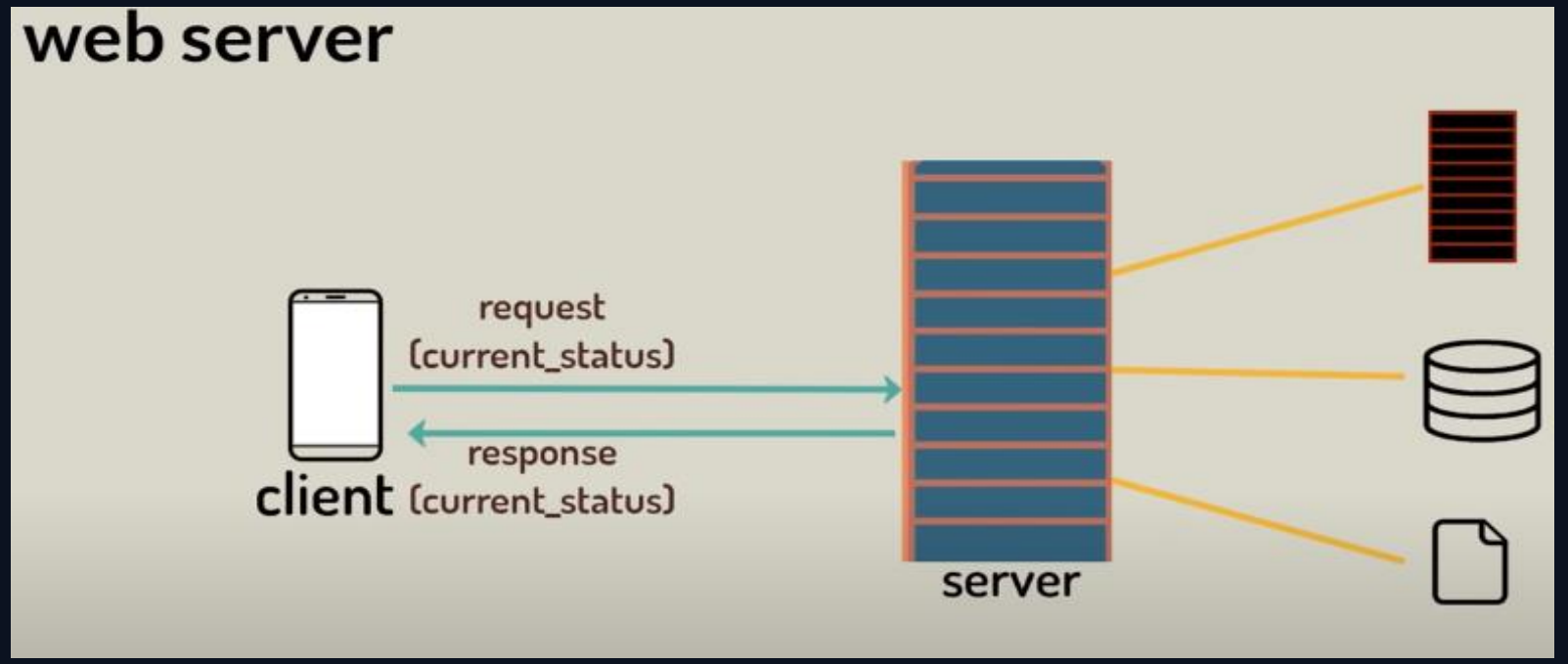
```
const http = require('http');

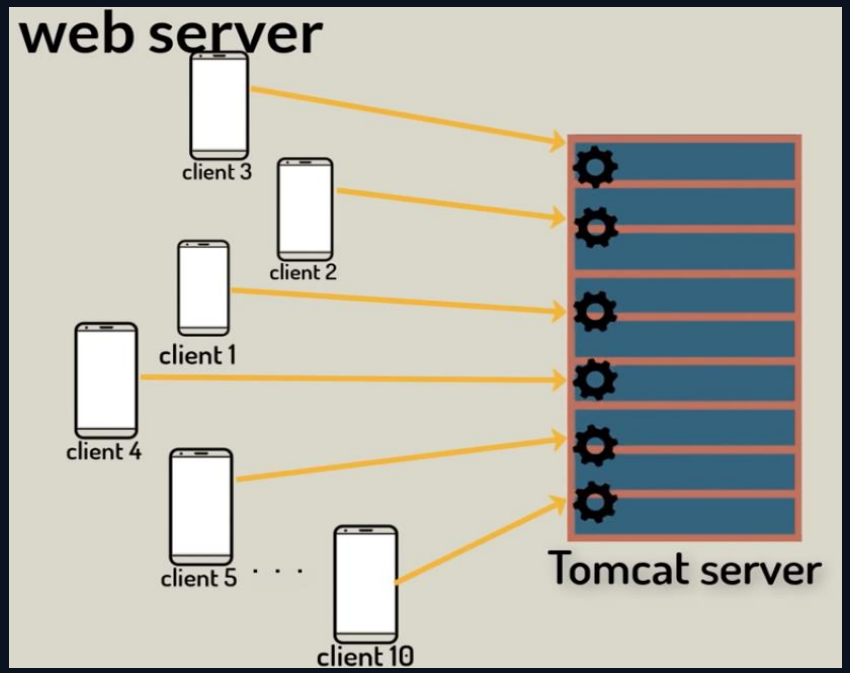
const server = http.createServer((req, res)=>{

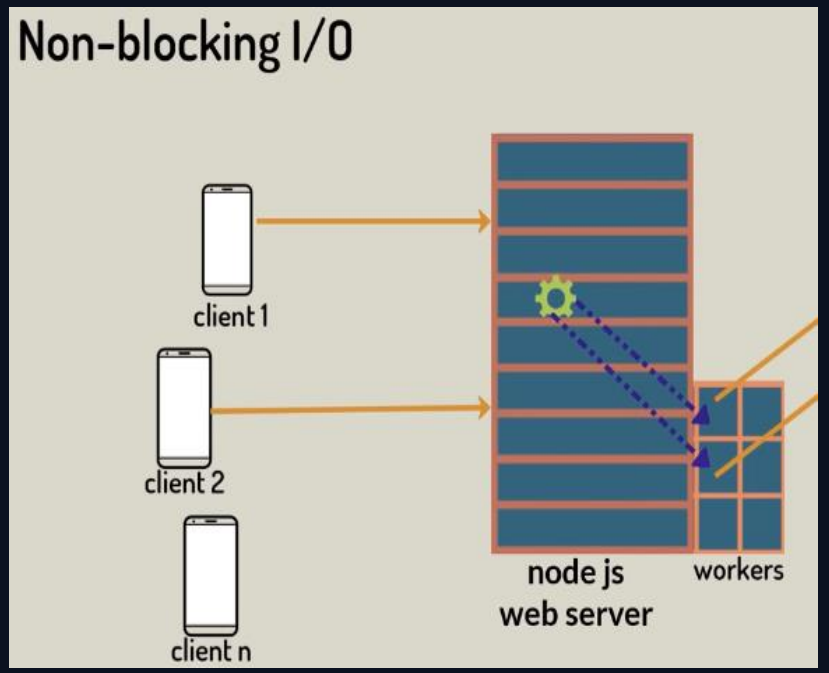
    res.write("Welcome to nodeJs !")
    res.end()

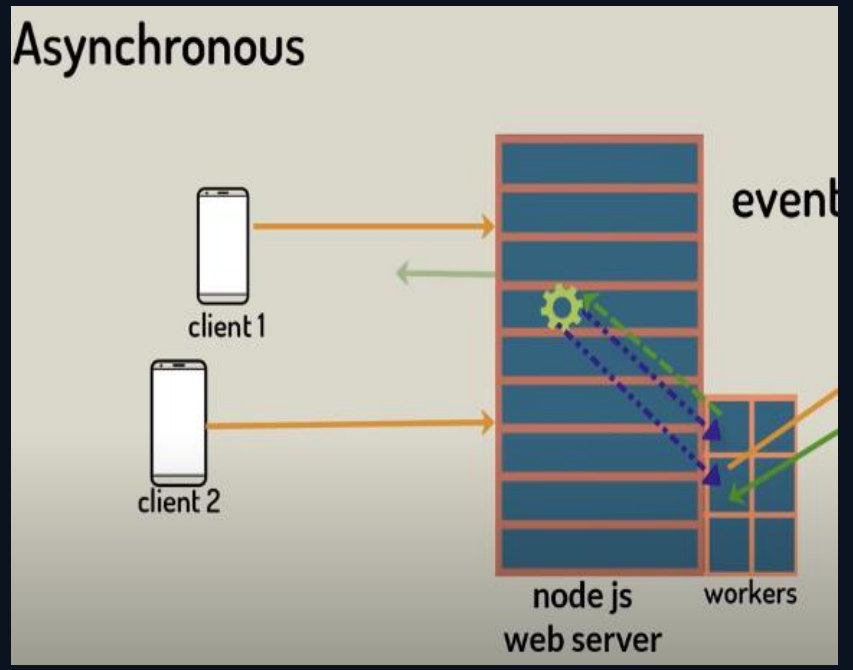
}).listen(9000)
```


client
server


web server
request
(current_status)
response
(current_status)
client
server


web server
client 3
client 2
client 1
client 4
client 5
client 10
Tomcat server


Non-blocking I/O
client 1
client 2
client n
node js
web server
workers


Asynchronous
client 1
client 2
node js
web server
workers

Thank You !